LLM APPLICATION CAPSTONE · PROJECT REPORT

SentinelAI: An Intelligent SOC Incident Triage Copilot

An evidence-grounded, citation-checked triage assistant for junior SOC analysts

github.com/ksrddd/sentinel-ai-llm-capstone · gemma-4-E4B-it via the class proxy, temperature 0 · retrieval results 2026-10-16, end-to-end results 2026-10-17

0.679 → 0.750
Recall@3 on held-out test, BM25 baseline → selected hybrid

0.696 → 0.729
MRR@5 on held-out test, same comparison

0.615 → 0.846
End-to-end severity accuracy, round r3 on its blind holdout (v4)

100%
Valid structured output and valid citations in all 18 round/set runs

Abstract

SentinelAI helps junior Security Operations Center (SOC) analysts triage security alerts. It extracts schema-validated facts from an alert, removes any value that is not literally in the alert, lets a bounded tool-calling agent search 19 original playbooks and past incidents, computes severity with a documented deterministic matrix, and drafts an incident report whose citations must come from the retrieved playbooks. On a held-out test split of 16 labelled queries, the retrieval configuration chosen on the dev split (hybrid-sbert-section-a0.3) raised Recall@3 from 0.679 to 0.750 and MRR@5 from 0.696 to 0.729 over a naive BM25 baseline. End-to-end, three improvement rounds were each measured on a freshly written 16-alert holdout set; the last round raised blind severity accuracy from 0.615 to 0.846. All sets are small and synthetic, so the report states the uncertainty and the remaining failures rather than a single headline number.

1. Problem and who it is for

Tier-1 analysts receive many alerts in inconsistent formats (JSON from an EDR, syslog lines from a VPN, one-line SIEM messages). Junior analysts make three recurring mistakes: (a) they confuse alerts that look alike but need different playbooks, such as a brute-force burst versus a successful login right after that burst; (b) they fill missing details with guesses; and (c) they assign severity inconsistently, so two analysts give the same alert different priorities. A generic chat-with-documents assistant makes (b) and (c) worse, because a language model will confidently invent hostnames, citations and priorities.

> **Design principle**
>
> **The model handles language; anything that must be reproducible or auditable is done by code and checked against evidence.** SentinelAI proposes a triage for a human to review; it never blocks, isolates, scans or executes anything.

- **Primary users:** junior SOC analysts who need a consistent first-pass triage with the right playbook.
- **Secondary users:** students learning incident triage, and instructors who want to see why a severity was assigned (the score breakdown and agent trace are shown in the dashboard).
- **Domain and data:** a fictional company ("Kestrel Analytics") with 19 original playbooks (PB-001..PB-019), an asset catalogue, 12 sample alerts and a synthetic incident history. None of it overlaps with the course reference app, whose corpus is the course's own software stack, so the reference app cannot answer these questions. IPs use RFC 5737 documentation ranges.

Objectives. O1 extract incident fields without inventing values; O2 retrieve the correct playbook and cite only what was retrieved; O3 let an agent choose tools safely; O4 compute severity deterministically; O5 measure each of these against a baseline and report it honestly.

2. Design

2.1 Architecture

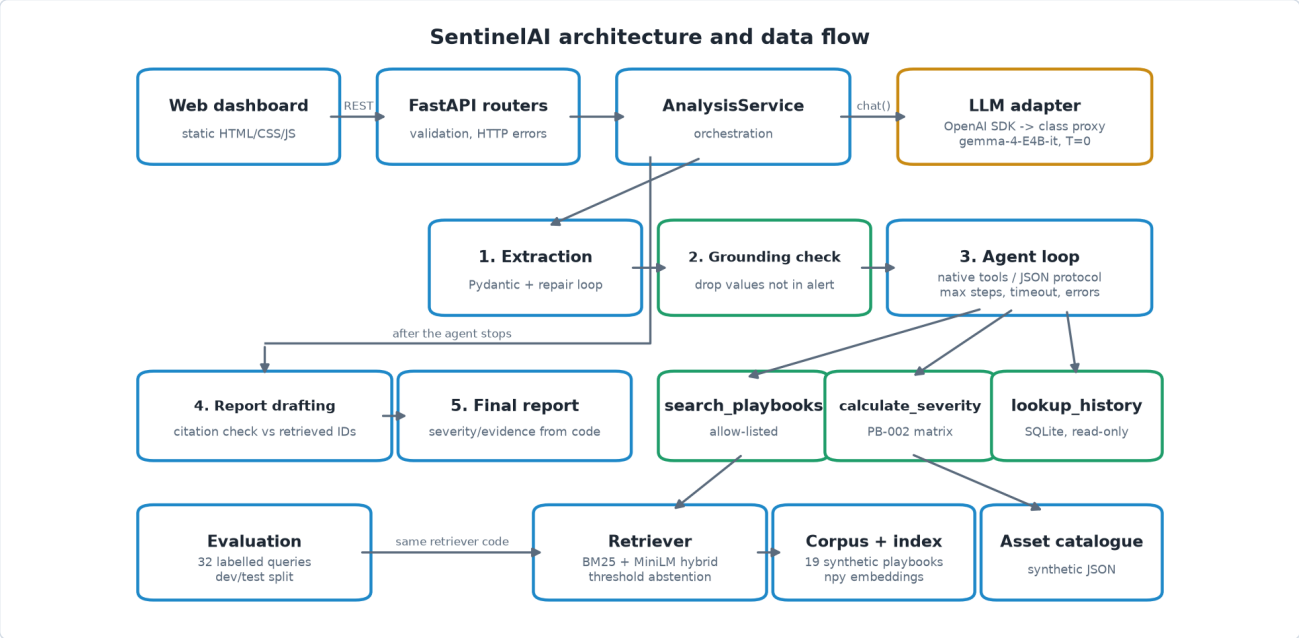


Figure 1. Layers and request flow. Green boxes are deterministic safety components.

The code follows the course's layering: routers (*app/api*) only validate HTTP input and map errors (422 invalid input, 400 malformed IDs, 502 failed model validation, 503 missing proxy configuration); services (*app/services*) hold the logic; repositories (*app/repositories*) wrap SQLite with parameterised SQL and the JSON asset catalogue; one LLM adapter (*app/llm/client.py*) is the only code that talks to the model, so the whole pipeline can be tested with a scripted fake client.

2.2 Request flow for POST /api/analyze

1. **Extract** - Gemma returns JSON validated by the *ExtractedIncident* Pydantic model (enumerated incident types, IP validation, bounded lists); up to 3 attempts, each retry receives the validator error.
2. **Ground** - every extracted value (host, IPs, user, timestamp, evidence, indicators) must appear verbatim in the alert, otherwise it is removed and listed as a limitation; prompt-injection phrases are detected and force human review.
3. **Agent** - the model chooses among three read-only tools (search_playbooks, calculate_severity, lookup_incident_history) for at most 6 model calls / 150 s.
4. **Enforce** - if the agent skipped retrieval or called calculate_severity with factors that differ from the grounded extraction, the orchestrator runs the tool itself and records an enforced step in the trace.
5. **Report** - Gemma drafts the summary, interpretation and actions; citations must be a subset of the retrieved IDs (bounded repair, else 502). Severity, evidence and citation metadata are filled in by code.
6. **Serve** - the result is stored in SQLite and returned to the dashboard, which shows the severity breakdown, cited actions (click to open the playbook), retrieved passages and the agent trace.

2.3 Proxy configuration and safety

The OpenAI SDK reads OPENAI_API_KEY and OPENAI_BASE_URL from the environment (.env is git-ignored; placeholder values from .env.example are rejected); model gemma-4-E4B-it, temperature 0 for every call. The proxy probe (scripts/check_proxy.py) recorded: list models ok, chat ok, system role ok, json mode ok, native tools ok. Alert text, playbook excerpts and tool results are wrapped in data tags and described to the model as untrusted; tool names are allow-listed and arguments use Pydantic models with extra fields forbidden, bounded integers and a hostname regex. No tool can write, scan, execute or reach the network.

2.4 Course techniques used and why

Table 1. Techniques, where they are used and why. Family (5) - SQL, web search, code execution, MCP - was deliberately not used: a triage assistant that can reach out of its sandbox would add risk without helping the three target mistakes.

Course family	How it is used here	Why this choice
(1) Structured output + schema validation	Pydantic models for extraction, report draft and every tool argument; bounded repair loop.	Downstream code (grounding, severity) needs typed fields. A failed repair returns 502 instead of a half-valid report, so the analyst never sees fabricated structure.
(2) RAG with citations	Heading-aware chunks, hybrid BM25 + MiniLM retrieval, min_score abstention, citation check.	Triage guidance must come from the organisation's playbooks, not model memory. Citing only retrieved IDs makes every recommendation checkable.
(3) Retrieval evaluation	32 labelled queries, dev/test split, Recall@3, MRR@5, abstention accuracy, BM25 baseline; plus a live end-to-end evaluation.	Chunking, the hybrid weight and the min_score floor were chosen by measurement on dev, not by guess.
(4) Agent loop with tools	Native OpenAI tool calling (JSON-action fallback); allow-list; stop conditions: final answer, 6 steps, 150 s, 3 consecutive errors, duplicate-call refusal.	Different alerts need different investigations (history lookup is useful for repeat hosts, not for phishing). Hard bounds keep a 4B model from looping.
(6) Served interface	FastAPI REST API (analyze, search, severity, playbooks, history, eval) + static dashboard.	Analysts work in a UI; the REST layer also makes every component testable on its own.
Deterministic severity (not an LLM task)	PB-002 matrix: tier + confidence + 2*impact + 2 if privileged; tier from the asset catalogue.	Consistency is the point of objective O4; a generated severity would differ between runs and could be argued out of the model by text inside the alert.

3. Evaluation

3.1 Test sets

Table 2. Evaluation data (all hand-written and hand-labelled).

Set	Size	Used for	Labels
eval/datasets/retrieval_eval_v1.jsonl (dev)	16 (14 answerable)	tuning thresholds, choosing the retrieval configuration	relevant playbook IDs, incident type, severity + its factors, should-abstain, justification note
eval/datasets/retrieval_eval_v1.jsonl (test)	16 (14 answerable)	reported once for baseline and selected configuration; end-to-end test split	same
eval/datasets/e2e_holdout_v2/v3/v4.jsonl	3 x 16 alerts	one blind measurement per improvement round (each written before that round's change)	same fields, alert-style wording

Scenario mix of the 32 retrieval queries: ambiguous 1, answerable 17, false positive 2, insufficient evidence 2, out of domain 4, policy question 3, similar different playbook 3. The mix deliberately includes look-alike pairs, vague alerts and out-of-domain questions, because those are where juniors and naive retrieval fail. Automated tests check that every relevant ID exists, that each severity label agrees with PB-002 given its factors, and that no query shares a 6-word sequence with the corpus (so a query cannot be answered by copied wording).

3.2 Metrics

- **Recall@3** = |relevant in top 3| / |relevant|; **MRR@5** = 1 / rank of the first relevant document in the top 5 (0 if none) - the primary pair, averaged over answerable queries. Precision@3 and Hit@1 are secondary.
- **Abstention accuracy** over all queries: correct iff the retriever abstains exactly when no relevant document exists. This is what the min_score floor controls.
- **End-to-end** (live Gemma): valid structured output rate, incident-type accuracy, severity accuracy (on alerts with a severity label), citation validity (every cited ID was retrieved), relevant-citation rate (a labelled relevant playbook is cited), undetermined-flagged rate, native tool-calling rate.

3.3 Baselines and protocol

- **Retrieval baseline A:** BM25 over fixed 120-word chunks - the naive first implementation. Candidates: BM25 with section chunks; dense LSA; dense MiniLM; hybrids with alpha in {0.3, 0.5, 0.7}.

- **min_score baseline:** no floor (abstain only if the top score is 0) versus a floor tuned on dev.
- **End-to-end baseline:** the first working pipeline (round r0), compared with each later round.
- **Protocol:** thresholds tuned on dev only; selection rule fixed in advance (max dev mean(Recall@3, MRR@5); tie-break dev abstention accuracy; then list order); test reported once. All model calls at temperature 0; a repeat run (r2b) checked that round differences are not sampling noise.

4. Results

4.1 Retrieval

Table 3. Baseline versus selected configuration on the held-out test split (16 queries, 14 answerable).

Held-out TEST	Recall@3	MRR@5	P@3	Hit@1	Abstention acc.
A: bm25-fixed (baseline)	0.679	0.696	0.238	0.643	0.875
B: hybrid-sbert-section-a0.3 (selected on dev)	0.750	0.729	0.262	0.643	0.875

Per-query reciprocal rank on test versus the baseline: 2 wins, 1 loss, 11 ties. On dev the selected configuration reached Recall@3 0.964 / MRR@5 0.952 versus 0.929 / 0.881 for the baseline.

Table 4. Before/after numbers for the three retrieval choices that differ from the naive default (computed from eval/results/retrieval_runs.json and per_example.csv).

Choice (naive default → chosen)	Split	Before	After	Reading
Chunking: fixed 120 words → one chunk per section (BM25 both)	test R@3 / MRR@5	0.679 / 0.696	0.750 / 0.750	Largest single gain; section headings carry the discriminative words.
Retriever: BM25 section → hybrid alpha 0.3	test R@3 / MRR@5	0.750 / 0.750	0.750 / 0.729	Better on dev (selection), slightly lower MRR on test - see 5.3.
min_score: none → 0.317 (selected config)	dev abstention	0.875	1.000	Both out-of-domain dev queries now abstain; Recall@3 unchanged (0.964).
	test abstention	0.875	0.875	No net gain on test: one correct abstention gained, one false abstention (Q04) introduced; Recall@3 0.750 → 0.750.

Table 5. All configurations. Selection used the dev columns only; test columns are shown for transparency and were not used for any decision.

Configuration	chunks	dev R@3	dev MRR@5	dev abst.	test R@3	test MRR@5	test abst.
bm25-fixed	40	0.929	0.881	1.000	0.679	0.696	0.875
bm25-section	85	0.964	0.917	1.000	0.750	0.750	0.875
dense-lsa-fixed	40	0.786	0.857	0.875	0.750	0.750	0.875
dense-lsa-section	85	0.893	0.875	0.875	0.750	0.679	0.875
hybrid-lsa-section-a0.3	85	0.893	0.911	1.000	0.750	0.714	0.812
hybrid-lsa-section-a0.5	85	0.893	0.911	0.875	0.750	0.714	0.875
hybrid-lsa-section-a0.7	85	0.893	0.911	0.875	0.750	0.679	0.875
dense-sbert-fixed	40	0.929	0.893	1.000	0.964	0.845	1.000
dense-sbert-section	85	0.857	0.800	1.000	0.821	0.854	1.000
hybrid-sbert-section-a0.3	85	0.964	0.952	1.000	0.750	0.729	0.875
hybrid-sbert-section-a0.5	85	0.929	0.952	1.000	0.821	0.762	0.938
hybrid-sbert-section-a0.7	85	0.929	0.952	1.000	0.821	0.809	1.000

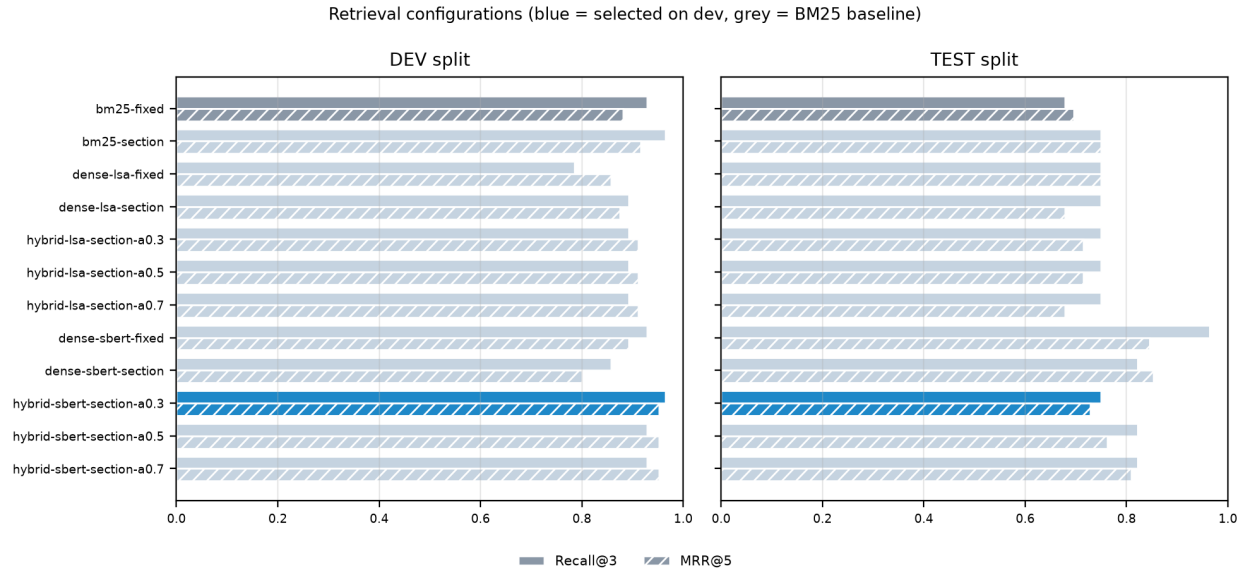


Figure 2. Recall@3 and MRR@5 per configuration on dev and test.

4.2 End-to-end application (live Gemma)

Every labelled alert is sent through the full pipeline (python -m eval.run_e2e_eval). The first run (r0) showed that Gemma hedged alert confidence to 'medium' and under-rated impact, which pulled severity down. Three rounds of changes followed (docs/EXPERIMENT_LOG.md). Before each round a new 16-alert holdout set was written and labelled, so each change has exactly one measurement on data it was not designed from.

Table 6. Each round measured on the holdout set written before it (before → after). This is the unbiased reading of the end-to-end improvements.

Round	Change	Blind set	Severity acc.	Relevant citation
r1	explicit confidence/impact definitions; insufficient-evidence search	holdout v2 (16)	0.462 → 0.615	0.750 → 0.812
r2	evidence kind → deterministic confidence; retry search after abstention	holdout v3 (16)	0.923 → 0.923	0.812 → 0.938
r3	yes/no 'malicious outcome confirmed' flag; label-only last-resort search	holdout v4 (16)	0.615 → 0.846	1.000 → 1.000

Table 7. Test split (12 alerts, 7 with a severity label). Its r0 failures were inspected before r1, so the final column is not blind.

Metric (test split)	r0 baseline	r3 final
valid structured output	1.000	1.000
incident type	1.000	1.000
severity	0.571	1.000
citation validity	1.000	1.000
relevant citation	0.917	1.000
undetermined flagged	1.000	1.000
native tool calling	1.000	1.000

Structured output was valid in every run of every round; the repair loop was needed in 29 of 260 recorded pipeline runs (almost always a second extraction attempt), and no report ever cited a playbook that had not been retrieved. The hard part was severity, which depends on the model's reading of confidence and impact. A repeat run of r2 (r2b) gave identical incident types, severities and factors, so the round-to-round differences are real at temperature 0. Pooled over all five sets, correct severities went from 40/51 (r2) to 43/51 (r3), but r3 also lost one severity and one citation case on holdout v3: prompt changes are not monotonic.

5. Analysis: successes and failure cases

5.1 Success case

Holdout alert W01 (malware on WS-FIN-114 with confirmed limited execution) shows the intended path: the agent called search_playbooks, lookup_incident_history and calculate_severity natively; extraction and report each validated on the first attempt; the factors (tier medium, confidence high, impact confirmed_limited) scored High as labelled, and the report cited PB-007, the correct malware playbook, in about 11 s. In offline tests with a scripted LLM, fabricated citations are repaired or rejected, invented hostnames are removed, and an alert that asks the model to 'run_shell' is refused as an unknown tool.

5.2 Failure case in detail: an under-triaged phishing alert (W11)

Alert W11 (holdout v4): expected High, predicted Low

`MAIL-GW-01 let through an email with a fake DocuSign link to 15 staff; two users entered their passwords on the page, which is still online.`

Table 8. Severity factors for W11, label versus prediction.

	Asset tier	Confidence	Impact	Score → severity
Label	high (3)	high (3)	confirmed_limited (2x2)	10 → High
SentinelAI (r3)	high (3)	low (1)	suspected (2x1)	6 → Low

The incident type (phishing) and the citations (PB-017, PB-005) were right, and the citation check passed. The error is in two judgements the model still makes: it classed a plain-English report as weak evidence (confidence low) and did not treat "two users entered their passwords" as a confirmed outcome (impact suspected instead of confirmed_limited). Because severity is computed from those factors, the deterministic matrix faithfully turned two wrong inputs into a Low. This is the most dangerous kind of error for a SOC - under-triage of a live credential compromise - and it shows the limit of the design: code makes severity consistent, but not correct when its inputs are wrong. The mitigation in place is that every report has requires_human_review set and shows the factor breakdown, so an analyst can see 'impact: suspected' next to an alert that says passwords were entered.

5.3 Other remaining failures

Table 9. Other end-to-end errors of the final round (r3) on the two most recent holdout sets.

Case	Expected	Predicted	Cause
W08 (v4)	unusual_outbound / Low	undetermined / Low	classified undetermined; the beacon pattern on a lab host was not recognised (safe: goes to human review, severity still right)
W10 (v4)	suspicious_dns / Medium	suspicious_dns / Low	C2-watchlist DNS match rated medium confidence → Medium became Low
V11 (v3)	phishing / Low	phishing / Informational	short phishing report classed vague; cited PB-015 (evidence) instead of PB-017
V16 (v3)	command_injection / High	command_injection / Medium	confirmed command injection kept at impact suspected → High became Medium

Table 10. Remaining retrieval test failures of the selected configuration.

Query	Scenario	Relevant	Top-5 (selected)	Cause (manual analysis)
Q02	answerable	PB-004	PB-018, PB-013, PB-005, PB-006, PB-004	hostname LAB-TEST-07 matches the asset catalogue (PB-018), which outranks PB-004
Q04	answerable	PB-005	PB-016, PB-001, PB-018, PB-004, PB-015	no lexical overlap (city names) and a weak dense score → false abstention
Q20	similar different playbook	PB-017, PB-005	PB-017, PB-004, PB-012, PB-013, PB-003	second relevant document (PB-005) ranked below 3
Q26	policy question	PB-016	PB-001, PB-018, PB-005, PB-011, PB-006	vocabulary mismatch: 'cut off the network' vs 'isolate'
Q30	out of domain	none (abstain)	PB-005, PB-015, PB-017, PB-018, PB-001	generic business words score just above the abstention floor

Honest reading

- Every retrieval configuration dropped from dev to test, so the dev split was easier. With 14 answerable test queries, one query moves Recall@3 by about 0.07; the hybrid's gain over the baseline (2 wins, 1 loss) is not statistically strong.
- dense-sbert-fixed was the best configuration on test (R@3 0.964, MRR@5 0.845) but was not chosen: switching now would mean selecting on the test set. We report it instead of re-selecting.
- The min_score floor helped on dev but was neutral on test (Table 4). One global threshold cannot separate 'generic business question' (Q30) from 'unusual wording of a real alert' (Q04).
- The test-split end-to-end numbers of 1.000 (Table 7) are not blind; the blind holdout rows in Table 6 (0.846 severity on v4) are the honest estimate.
- Most end-to-end errors are under-rating, not over-rating. For a triage tool that is the worse direction.

6. Limitations

- **Synthetic and small.** 19 short playbooks, 32 retrieval queries and 12-16 alerts per end-to-end set, written by the author. Results do not transfer to real SOC telemetry, and one case moves end-to-end accuracy by 6-20 points.
- **Label ambiguity.** Some errors (W10, H07) sit on the border between 'anomaly' and 'specific telemetry'; a second labeller would likely disagree on a few labels. No inter-annotator agreement was measured.
- **Model judgement inside a deterministic score.** Confidence and impact still come from Gemma; the matrix makes severity consistent but cannot correct wrong factors (Section 5.2).
- **Lossy grounding.** Literal substring matching drops correct paraphrases (safe but lossy). Injection detection is pattern-based; the real protection is that tools are read-only and allow-listed.
- **Single global abstention threshold** and a single-process SQLite store without authentication: the app is meant to run on localhost only.
- **Report quality not scored.** The summary and recommended actions are checked for valid citations but not graded for usefulness.

7. Next steps

Ordered by expected value; each item names how it would be measured, on a new holdout set so that the measurement stays blind.

1. **Fix under-triage first.** Ask for the impact evidence as a quoted span from the alert and map phrases such as 'entered their passwords' or 'process executed' to confirmed impact in code. Measure: severity accuracy and the number of under-rated cases on holdout v5.
2. **Query cleaning for retrieval.** Strip hostnames and IPs from search queries (Q02) and add a small synonym list (isolate/disconnect/cut off, Q26). Measure: Recall@3 and MRR@5 on a new dev set, then test.
3. **Better abstention.** Replace the single score floor with a margin rule (top-1 minus top-2) or a per-category floor. Measure: abstention accuracy, separately for false and missed abstentions.
4. **A larger, second-labeller dataset.** 100+ queries labelled by two people, with Cohen's kappa, so differences of a few points become meaningful.
5. **LLM-as-judge for report quality,** calibrated against human ratings on 20 reports before trusting it.
6. **Analyst feedback loop** in the dashboard (accept/override severity) to collect real error cases.

Reproducibility and references

Retrieval: `python -m eval.run_retrieval_eval` (CPU, about 1 min); end-to-end: `python -m eval.run_e2e_eval --split test --round r3_outcome_flag_label_fallback`; figures: `python -m scripts.make_figures`; this report: `python -m scripts.build_report`; offline tests: `pytest` (mocked LLM). Python 3.13.12; LSA seed 42; BM25 k1=1.5, b=0.75; embeddings sentence-transformers/all-MiniLM-L6-v2; dataset sha256/16 ef03063861a8e6be. Per-example results: eval/results/per_example.csv and eval/results/rounds/.

References: Nat-D/kmitl-llm-app-demo (course reference app; general ideas only - thin LLM adapter, min_score floor, router/service/repository layering - no code, corpus or evaluation data copied); Robertson & Zaragoza (2009) BM25; Reimers & Gurevych (2019) Sentence-BERT; Deerwester et al. (1990) LSA; RFC 5737.